

Chapter 6

Managing Local Users and Groups

➤ What is a User?

- A user account is used to provide security boundaries between different people and programs that can run commands.
- Users have user names to identify them to human users and make them easier to work with.
- Internally, the system distinguishes user accounts by the unique identification number assigned to them, the **user ID or UID**.

You can use the **id** command to show information about the currently logged-in user

```
[user01@host ~]$ id
uid=1000(user01) gid=1000(user01) groups=1000(user01)
context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

```
[user01@host]$ id user02
uid=1002(user02) gid=1001(user02) groups=1001(user02)
context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

User File in Linux – # cat /etc/passwd

About /etc/passwd File

```
1 user01:2x:31000:41000:5User One:6/home/user01:7/bin/bash
```

- 1 Username for this user (**user01**).
- 2 The user's password used to be stored here in encrypted format. That has been moved to the **/etc/shadow** file, which will be covered later. This field should always be **x**.
- 3 The UID number for this user account (**1000**).
- 4 The GID number for this user account's primary group (**1000**). Groups will be discussed later in this section.
- 5 The real name for this user (**User One**).
- 6 The home directory for this user (**/home/user01**). This is the initial working directory when the shell starts and contains the user's data and configuration settings.
- 7 The default shell program for this user, which runs on login (**/bin/bash**). For a regular user, this is normally the program that provides the user's command-line prompt. A system user might use **/sbin/nologin** if interactive logins are not allowed for that user.

What is a Group?

- A group is a collection of users that need to share access to files and other system resources.
- Groups can be used to grant access to files to a set of users instead of just a single user.
- Like users, groups have group names to make them easier to work with.
- Internally, the system distinguishes groups by the unique identification number assigned to them, the **group ID or GID**.

By default, systems use the **/etc/group** file to store information about local groups.

```
1group01: 2x: 310000: 4user01,user02,user03
```

- 1 Group name for this group (**group01**).
- 2 Obsolete group password field. This field should always be **x**.
- 3 The GID number for this group (**10000**).
- 4 A list of users who are members of this group as a supplementary group (**user01, user02, user03**). Primary (or default) and supplementary groups are discussed later in this section.

Managing Local User Accounts

- The **useradd 'username'** command creates a **new-user** named username.
- The **useradd --help** command displays the basic options that can be used to override the defaults. In most cases, the same options can be used with the **usermod** command to modify an **existing user**.
- Some defaults, such as the range of **valid UID** numbers and **default password aging rules**, are read from the **/etc/login.defs** file

Modifying Existing Users from the Command Line

- The **usermod --help** command displays the basic options that can be used to modify an account.

usermod options:	Usage
-c, --comment COMMENT	Add the user's real name to the comment field.
-g, --gid GROUP	Specify the primary group for the user account.
-G, --groups GROUPS	Specify a comma-separated list of supplementary groups for the user account.
-a, --append	Used with the -G option to add the supplementary groups to the user's current set of group memberships instead of replacing the set of supplementary groups with a new set.
-d, --home HOME_DIR	Specify a particular home directory for the user account.
-m, --move-home	Move the user's home directory to a new location. Must be used with the -d option.
-s, --shell SHELL	Specify a particular login shell for the user account.
-L, --lock	Lock the user account.
-U, --unlock	Unlock the user account.

Deleting Users from the Command Line

- The **userdel username** command removes the details of username from **/etc/passwd**, but leaves the user's home directory intact.
- The **userdel -r username** command removes the details of username from **/etc/passwd** and also deletes the user's home directory.

Setting Passwords from the Command Line

- The **# passwd username** command sets the initial password or changes the existing password of username.
- The **root** user can set a **password to any value**. A message is displayed if the password does not meet the minimum recommended criteria, but is followed by a prompt to retype the new password and all tokens are updated successfully.

```
[root@host ~]# passwd user01
Changing password for user user01.
New password: redhat
BAD PASSWORD: The password fails the dictionary check - it is based on a
dictionary word
Retype new password: redhat
passwd: all authentication tokens updated successfully.
[root@host ~]#
```

UID Ranges

Specific UID numbers and ranges of numbers are used for specific purposes by Red Hat Enterprise Linux.

- **UID 0** is always assigned to the **superuser account, root**.
- **UID 1-200** is a range of "**system users**" assigned **statically to system processes by Red Hat**.
- **UID 201-999** is a range of "**system users**" used by system processes that do not own files on the file system. **They are typically assigned dynamically** from the available pool when the software that needs them is installed. Programs run as these "unprivileged" system users in order to limit their access to only the resources they need to function.
- **UID 1000+** is the range available for assignment to **regular users**.

➤ Managing Local Groups

A group **must exist** before a user can be added to that group. Several command-line tools are used to manage local group accounts.

The **groupadd** command **creates groups**.

The **-g** option specifies **a particular GID** for the group to use.

```
[user01@host ~]$ sudo groupadd -g 10000 group01
[user01@host ~]$ tail /etc/group
...output omitted...
group01:x:10000:
```

Modifying Existing Groups from the Command Line

- The **groupmod** command changes the properties of an existing group.
- The **-n** option specifies a **new name for the group**.

```
[user01@host ~]$ sudo groupmod -n group0022 group02
[user01@host ~]$ tail /etc/group
...output omitted...
group0022:x:988:
```

- The **-g** option specifies a **new GID**.

```
[user01@host ~]$ sudo groupmod -g 20000 group0022
[user01@host ~]$ tail /etc/group
...output omitted...
group0022:x:20000:
```

Deleting Groups from the Command Line

- The **groupdel** command **removes groups**.

```
[user01@host ~]$ sudo groupdel group0022
```

Changing Group Membership from the Command Line

- The membership of a group is controlled with user management. Use the **usermod -g** command to **change a user's primary group**.

```
[user01@host ~]$ id user02
uid=1006(user02) gid=1008(user02) groups=1008(user02)
[user01@host ~]$ sudo usermod -g group01 user02
[user01@host ~]$ id user02
uid=1006(user02) gid=10000(group01) groups=10000(group01)
```

- Use the **usermod -aG** command to add a user to a **supplementary group**.

```
[user01@host ~]$ id user03
uid=1007(user03) gid=1009(user03) groups=1009(user03)
[user01@host ~]$ sudo usermod -aG group01 user03
[user01@host ~]$ id user03
uid=1007(user03) gid=1009(user03) groups=1009(user03),10000(group01)
```

Managing User Passwords

- Like `/etc/passwd`, each user has a line in the `/etc/shadow` file.
- A sample line from `/etc/shadow` with its nine colon-separated fields

```
1 user03: 2 $6$CSsX...output omitted...: 3 17933: 4 0: 5 99999: 6 7: 7 2: 8 18113: 9
```

- 1 Username of the account this password belongs to.
- 2 The *encrypted password* of the user. The format of encrypted passwords is discussed later in this section.
- 3 The day on which the password was last changed. This is set in days since 1970-01-01, and is calculated in the UTC time zone.
- 4 The minimum number of days that have to elapse since the last password change before the user can change it again.
- 5 The maximum number of days that can pass without a password change before the password expires. An empty field means it does not expire based on time since the last change.
- 6 Warning period. The user will be warned about an expiring password when they login for this number of days before the deadline.
- 7 Inactivity period. Once the password has expired, it will still be accepted for login for this many days. After this period has elapsed, the account will be locked.
- 8 The day on which the account expires. This is set in days since 1970-01-01, and is calculated in the UTC time zone. An empty field means it does not expire on a particular date.
- 9 The last field is usually empty and is reserved for future use.

Configuring Password Aging

- The following diagram relates the relevant password aging parameters, which can be adjusted
- using the **chage** command to implement a password aging policy

```
[user01@host ~]$ sudo chage -m 0 -M 90 -W 7 -I 14 user03
```

- The preceding **chage** command uses the **-m**, **-M**, **-W**, and **-I** options to set the **minimum age**, **maximum age**, **warning period**, and **inactivity period** of the user's password, respectively.
- The **chage -d 0 user03** command forces the user03 user to update its password on the **next login**.
- The **chage -l user03** command displays the password aging **details of user03**.
- The **chage -E 2019-08-05 user03** command causes the user03 user's account to expire on **2019-08-05 (in YYYY-MM-DD format)**.

- The **usermod** command can lock an account with the **-L** option.

```
[user01@host ~]$ sudo usermod -L user03
[user01@host ~]$ su - user03
Password: redhat
su: Authentication failure
```

- The preceding **usermod** command uses the **-e** option to set the account expiry date for the given user account. The **L** option locks the user's password.
- If the employee returns, the account can later be unlocked with **usermod -U**.

```
[user01@host ~]$ sudo usermod -L -e 2019-10-05 user03
```

- A common solution to this situation is to set the user's login shell to **/sbin/nologin**. If the user
- attempts to log in to the system directly, the **nologin** shell closes the connection.

```
[user01@host ~]$ usermod -s /sbin/nologin user03
[user01@host ~]$ su - user03
Last login: Wed Feb  6 17:03:06 IST 2019 on pts/0
This account is currently not available.
```

Gaining Superuser Access

- Most operating systems have some sort of **superuser**, a user that has all power over the system. In Red Hat Enterprise Linux this is the **root** user.

Switching Users

- The **su** command allows users to **switch to a different user account**.
- If you run **su** from a **regular user account**, you will be **prompted for the password** of the account to which you want to switch.
- When **root** runs **su**, you **do not need to enter the user's password**.
- If you **omit the username**, the **su** or **su -** command attempts to switch to **root by default**.

```
[user01@host ~]$ su - user02
Password:
[user02@host ~]$
```

```
[user01@host ~]$ su -
Password:
[root@host ~]#
```

Configuring Sudo

- The main configuration file for sudo is **/etc/sudoers**.
- To avoid problems if multiple administrators try to edit it at the same time, it should only be edited with the special **visudo** command.
- To enable full sudo access for the user user01, you could create **/etc/sudoers.d/user01**

```
user01  ALL=(ALL)  ALL
```

- To enable full **sudo** access for the group group01, you could create **/etc/sudoers.d/group01**

```
%group01  ALL=(ALL)  ALL
```

- It is also possible to set up **sudo** to allow a user to run commands as another user **without entering their password**

```
ansible  ALL=(ALL)  NOPASSWD:ALL
```